QuillAudits

Audit Report
September, 2021

between.network

Contents

Scope of Audit

The scope of this audit was to analyze and document the BetweenNetwork Multi-Cross-Chain Token Bridge smart contract codebase for quality, security, and correctness.

Checked Vulnerabilities

We have scanned the smart contract for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- DoS with Block Gas Limit
- Transaction-Ordering Dependence
- Use of tx.origin
- Exception disorder
- Gasless send
- Balance equality
- Byte array
- Transfer forwards all gas
- ERC20 API violation
- Malicious libraries
- Compiler version not fixed
- Redundant fallback function
- Send instead of transfer
- Style guide violation
- Unchecked external call
- Unchecked math
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contract, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments match logic and expected behaviour.
- Token distribution and calculations are as per the intended behaviour mentioned in the whitepaper.
- Implementation of ERC-20 token standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods and tools were used to review all the smart contracts.

Structural Analysis
In this step we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.
SmartCheck.

Static Analysis
Static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step a series of automated tools are used to test security of smart contracts.

Code Review / Manual Analysis
Manual Analysis or review of code was done to identify new vulnerability or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of automated analysis were manually verified.

Gas Consumption
In this step we have checked the behaviour of smart contracts in production. Checks were done to know how much gas gets consumed and possibilities of optimization of code to reduce gas consumption.

Tools and Platforms used for Audit
Remix IDE, Truffle, Truffle Team, Ganache, Solhint, Mythril, Slither, SmartCheck.

Issue Categories

Every issue in this report has been assigned with a severity level. There are four levels of severity and each of them has been explained below.

High severity issues

A high severity issue or vulnerability means that your smart contract can be exploited. Issues on this level are critical to the smart contract's performance or functionality and we recommend these issues to be fixed before moving to a live environment.

Medium level severity issues

The issues marked as medium severity usually arise because of errors and deficiencies in the smart contract code. Issues on this level could potentially bring problems and they should still be fixed.

Low level severity issues

Low level severity issues can cause minor impact and or are just warnings that can remain unfixed for now. It would be better to fix these issues at some point in the future.

Informational

These are severity four issues which indicate an improvement request, a general question, a cosmetic or documentation error, or a request for information. There is low-to-no impact.

Number of issues per severity

Type	High	Medium	Low	Informational
Open	0	0	0	0
Acknowledged	0	1	1	1
Closed	1	4	5	3

Introduction

During the period of **August 20, 2021 to August 25, 2021** - QuillAudits Team performed a security audit for BetweenNetwork smart contracts.

The code for the audit was taken from following the official link: https://github.com/BetweenNetwork/bridge-upgradable

Note	Date	Commit ID
Version 1	August 22	d7304bb9023f08502b02b1d0f3d1c09be687d04a
Version 2	September 06	319dc7dcbfc30df074a7d5d71c7d3148416a68a2

Issues Found – Code Review / Manual Testing

High severity issues

1. **The length of validators array does not change after removing its validators**

   Description
   The validators.length does not change after a validator is removed via removeValidator() function, which has an impact on the contract's business logic. This also implies that the following operation will not function as expected:

```
if (required > validators.length) {
        changeRequirement(validators.length);
}
```

   Remediation
   We recommend adding the validators.pop function to the removeValidator().

   Status: Fixed

Medium severity issues

2. **Centralization Risks**

   Description
   The role Governance has the authority to update the critical settings:
   - updateDepositFee()
   - updateListingFee()
   - updateMultiplierReward()
   - updateRewardToken()
   - updateWithdrawFee()

Remediation
We advise the client to handle the governance account carefully to avoid any potential hack. We also recommend the client to consider the following solutions:

- with reasonable latency for community awareness on privileged operations;
- Multisig with community-voted 3rd-party independent co-signers;
- DAO or Governance module increasing transparency and community involvement;

Status: Acknowledged by the Auditee
The Auditee confirmed that the Multisig will be utilized in the next update.

3. Lack of event emissions

Line	Code

426	function updateMultiplierReward(uint16 _newMultiplier) external onlyGovernance {
	require(_newMultiplier > 0, "updateMultiplierReward:: Multiplier can not be Zero");
	multiplier = _newMultiplier;
	}

Description
The missing event makes it difficult to track off-chain liquidity fee changes. An event should be emitted for significant transactions calling the following functions.

Remediation
We recommend emitting an event to log the update of the multiplier variable.

Status: Fixed

4. Loops in the Contract are extremely costly

Description

The for loops in the entire codebase includes state variables .length of a non-memory array, in the condition of the for loops. As a result, these state variables consume a lot more extra gas for every iteration of the for loop.

Remediation

We recommend using a local variable instead of a state variable .length in a loop for the entire codebase. For example:

```
 _validatorLength = _validator.length
for (uint i=0; i < _validatorLength; i++) {
.....
}
```

Status: Fixed

5. Divisions performed before multiplication

Description

Due to the fact that Solidity truncates when dividing, performing a division before multiplication can result in truncated amounts being amplified by future calculations. There are a few instances in the codebase where division is performed mid-calculation. For examples:
L261: platformFee = _amount.mul(depositFee).div(MAX_PLATFORM_FEE).mul(multiplier)
L272: platformFee = _amount.mul(withdrawFee).div(MAX_PLATFORM_FEE).mul(multiplier)

Remediation

Being mindful of overflows, consider changing the order of operations where possible such that truncating steps are performed last to minimize any unnecessary loss of precision. We recommend changing the order of the divisions in the contract.

Status: Fixed

6. transfer() is being utilized

Description

Low-level transfer() function has been found to be used in the contract at line 215:
payable(beneficiary).transfer(listingFee);

Due to the fact that .transfer() and .send() forward exactly 2,300 gas to the recipient. This hardcoded gas stipend aimed to prevent reentrancy vulnerabilities, but this only makes sense under the assumption that gas costs are constant. Recently EIP 1884 was included in the Istanbul hard fork. One of the changes included in EIP 1884 is an increase to the gas cost of the SLOAD operation, causing a contract's fallback function to cost more than 2300 gas.

```
// bad
contract Vulnerable {
    function withdraw(uint256 amount) external {
        // This forwards 2300 gas, which may not be enough if the recipient
        // is a contract and gas costs change.
        msg.sender.transfer(amount);
    }
}

// good
contract Fixed {
    function withdraw(uint256 amount) external {
        // This forwards all available gas. Be sure to check the return value!
        (bool success, ) = msg.sender.call.value(amount)("");
        require(success, "Transfer failed.");
    }
}
```

Remediation

The auditee needs to ensure that the beneficiary is not a contract .On the other hand, it's recommended to stop using .transfer() and .send() and instead use .call().

Status: Fixed

Low level severity issues

7. Missing Range Check for Input Variable

Line	Code

379	function updateListingFee(uint256 _newListingFee) external onlyGovernance {
	require(_newListingFee > 0, "UpdateListingFee:: _newListingFee can not be Zero");
	require(listingFee != _newListingFee, "UpdateListingFee:: New Listing Fee can not be same as Listing Fee");
	listingFee = _newListingFee;
	emit ListingFeeUpdated(listingFee, _newListingFee);
	}

Description

The role can set the listingFee state variables arbitrarily large or small causing potential risks in fees.

Remediation

We recommend setting ranges and check the _newListingFee input variables.

Status: Acknowledged by the Auditee

8. Uninformative revert messages in require statements

The following instances in the codebase where the require statement have ambiguous or imprecise error messages.

L202: require(_required > 0, "ChangeRequirement:: Validator can not be Zero");
L387: require(_newDepositFee > 0, "UpdateDepositFee:: _newListingFee can not be Zero");
L388: require(_newDepositFee <= MAX_PLATFORM_FEE, "UpdateDepositFee:: Invalid withdraw fee ");
L421: require(_newRewardToken != address(0), "UpdateRewardToken:: Renounce Reward Token");

Uninformative error messages greatly damage the overall user experience, thus lowering the system's quality. Consider not only fixing the specific instance mentioned above, but also reviewing the entire codebase to make sure every error message is informative and user-friendly

Status: Fixed

9. Comparison to boolean constants

Description

We have found that majority boolean variables in the contract are compared to true or false, for example:

- require(isBridgePaused == false,CreateBridgePair:: Bridge Paused, can not add new token)
- require(depositWallets[msg.sender]._isBanned == false,TapIn:: Your Wallet Banned)
- require(isTapInPaused == false,TapIn:: TapIn Paused, can not Deposit)

While those boolean variables can be used directly and do not need to be compared to true or false.

Remediation

We recommend removing the equality to the boolean constant in the entire codebase.

Status: Fixed

10. Lack of error message for require() functions

Description

The require statements in the entire code base should include error messages that detail the cause of errors. For examples:

- require(_address != address(0));
- require(isValidator[validator]);
- require(validatorCount <= MAX_VALIDATOR_COUNT && _required <= validatorCount && _required != 0 && validatorCount != 0);

Remediation

We recommend adding message errors for require() statements for the entire BetweenNetwork's codebase.

Status: Fixed

11. **An incorrect variable in the WithdrawFeeUpdated event emission**

Description
L399: emit WithdrawFeeUpdated(depositFee, _newWithdrawFee);

The above emit function updates a wrong variable of the withdrawFee.

Remediation
We recommend changing the depositFee to withdrawFee

Status: Fixed

12. **Lack of Zero address validation in initialize() function**

Description
To favor explicitness, consider adding a check for all functions that are taking address parameters in the entire codebase. Although most of the functions throughout the codebase properly validate function inputs, there are some instances of functions that do not. One example is:
- initialize() - does not check for zero address

Remediation
Consider implementing require statements where appropriate to validate all user-controlled input, including governance functions, to avoid the potential for erroneous values to result in unexpected behaviors or wasted gas.

Status: Fixed

Informational

13. An incorrect variable in the WithdrawFeeUpdated event emission

Description
Throughout BetweenNetwork's codebase, there are a few typos in the code and in the comments. We list them here:

- _depositeTxHash should be _depositedTxHash
- _withdrawlStatus should be _withdrawalStatus
- DepositeTxHashs should be DepositTxHashes
- brdige should be bridge
- _withdraERC20 should be _withdrawERC20
- banUser should be bannedUser
- BanUser should be BannedUser
- unBanUser should be unbannedUser
- UnbanUser should be UnbannedUser

Status: Fixed

14. Conformance to Solidity naming conventions

Description
Deviations from the naming conventions were identified throughout the entire codebase. Taking into consideration how much value a consistent coding style adds to the project's readability.

Functions other than constructors should use mixedCase. Examples: getBalance, transfer, verifyOwner, addMember, changeOwner.
L182: addvalidator() should be addValidator()
L173: MultiSigWallet() should be multiSigWallet()

We advise the Auditee to adhere to the naming convention rules for the entire codebase.

Status: Fixed

15. Variables declared as uint instead of uint256

Description

To favor explicitness, consider changing all instances of uint into uint256 in the entire codebase. For example:

L14: using SafeMathUpgradeable for uint;
L107: event RequirementChange(uint required);
L165: modifier validRequirement(uint validatorCount, uint _required) {
L173: function MultiSigWallet(address[] memory _validator, uint _required) internal validRequirement(_validator.length, _required) {
L174: for (uint i=0; i<_validator.length; i++) {
L190: for (uint i=0; i<validators.length - 1; i++)
L201: function changeRequirement(uint _required) internal validRequirement(validators.length, _required) {

Status: Fixed

16. Missing docstrings

Description

It is extremely difficult to locate any contracts or functions, as they lack documentation. One consequence of this is that reviewers' understanding of the code's intention is impeded, which is significant because it is necessary to accurately determine both security and correctness.

They are additionally more readable and easier to maintain when wrapped in docstrings. The functions should be documented so that users can understand the purpose or intention of each function, as well as the situations in which it may fail, who is allowed to call it, what values it returns, and what events it emits.

Status: Acknowledged by the Auditee

Functional test

Function Names	Testing results
initialize	Passed
addvalidator	Passed
removeValidator	Passed
createBridgePair	Passed
tapIn	Passed
tapOut	Passed
suspendToken	Passed
activeSuspendToken	Passed
banUser	Passed
unBanUser	Passed
pauseTapIn	Passed
unPauseTapIn	Passed
pauseTapOut	Passed
unPauseTapOut	Passed
updateListingFee	Passed
updateDepositFee	Passed
updateWithdrawFee	Passed
updateBeneficiary	Passed
pauseBridge	Passed
unPauseBridge	Passed

Function Names	Testing results
updateRewardToken	Passed
updateMultiplierReward	Passed
renounceRewardToken	Passed
validatorSignMessage	Passed

Automated Testing

Slither

```
INFO:Detectors:
Reentrancy in BetweenNetwork.tapIn(address,uint256) (BetweenNetwork.sol#222-235):
        External calls:
        - depositTokensAndPlatformFee(_homeERC20,_amount) (BetweenNetwork.sol#231)
                - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.
sol#92)
                - IERC20Upgradeable(_homeERC20).safeTransferFrom(msg.sender,beneficiary,platformFee) (BetweenNetwork.sol#263)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
                - IERC20Upgradeable(rewardToken).safeTransfer(msg.sender,platformFee) (BetweenNetwork.sol#266)
                - IERC20Upgradeable(_homeERC20).safeTransferFrom(msg.sender,address(this),_amount.sub(platformFee)) (BetweenNetwork.sol#268)
        External calls sending eth:
        - depositTokensAndPlatformFee(_homeERC20,_amount) (BetweenNetwork.sol#231)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
        State variables written after the call(s):
        - updateDeposit(lastDepositIndex,_amount,_homeERC20,trustedTokens[_homeERC20]._foreignERC20,block.number,block.timestamp) (BetweenNetwork.sol#233)
                - depositWallets[msg.sender]._index = _index (BetweenNetwork.sol#296)
                - depositWallets[msg.sender]._amount = _amount (BetweenNetwork.sol#297)
                - depositWallets[msg.sender]._homeChainId = homeChainId (BetweenNetwork.sol#298)
                - depositWallets[msg.sender]._foreignChainId = foreignChainId (BetweenNetwork.sol#299)
                - depositWallets[msg.sender]._user = msg.sender (BetweenNetwork.sol#300)
                - depositWallets[msg.sender]._homeERC20 = _homeERC20 (BetweenNetwork.sol#301)
                - depositWallets[msg.sender]._foreignERC20 = _foreignERC20 (BetweenNetwork.sol#302)
                - depositWallets[msg.sender]._depositedBlock = _depositedBlock (BetweenNetwork.sol#303)
                - depositWallets[msg.sender]._depositedTimeStamp = _depositedTimeStamp (BetweenNetwork.sol#304)
Reentrancy in BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[]) (BetweenNetwork.sol#237-258):
        External calls:
        - withdrawTokensAndPlatformFee(_withdrawERC20,_amount) (BetweenNetwork.sol#254)
                - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.
sol#92)
                - IERC20Upgradeable(_withdrawERC20).safeTransfer(beneficiary,platformFee) (BetweenNetwork.sol#274)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
                - IERC20Upgradeable(rewardToken).safeTransfer(msg.sender,platformFee) (BetweenNetwork.sol#277)
                - IERC20Upgradeable(_withdrawERC20).safeTransfer(msg.sender,_amount.sub(platformFee)) (BetweenNetwork.sol#279)
        External calls sending eth:
        - withdrawTokensAndPlatformFee(_withdrawERC20,_amount) (BetweenNetwork.sol#254)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
        State variables written after the call(s):
        - claimedWithdrawalsByOtherChainDepositId[_nonce] = true (BetweenNetwork.sol#255)
        - updateWithdraw(_nonce,_amount,_withdrawERC20,_depositedERC20,_txHash) (BetweenNetwork.sol#256)
                - withdrawalWallets[msg.sender]._index = _nonce (BetweenNetwork.sol#309)
                - withdrawalWallets[msg.sender]._amount = _amount (BetweenNetwork.sol#310)
                - withdrawalWallets[msg.sender]._homeChainId = homeChainId (BetweenNetwork.sol#311)
                - withdrawalWallets[msg.sender]._foreignChainId = foreignChainId (BetweenNetwork.sol#312)
                - withdrawalWallets[msg.sender]._user = msg.sender (BetweenNetwork.sol#313)
                - withdrawalWallets[msg.sender]._withdrawERC20 = _withdrawERC20 (BetweenNetwork.sol#314)
```

~/Quilhash/bridge-upgradable/01Code/bridge-upgradable-main_slither/contracts — -zsh

```
                - withdrawalWallets[msg.sender]._withdrawERC20 = _withdrawERC20 (BetweenNetwork.sol#314)
                - withdrawalWallets[msg.sender]._depositedERC20 = _depositedERC20 (BetweenNetwork.sol#315)
                - withdrawalWallets[msg.sender]._depositeTxHash = _txHash (BetweenNetwork.sol#316)
                - withdrawalWallets[msg.sender]._withdrawlStatus = true (BetweenNetwork.sol#317)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities
INFO:Detectors:
OwnableUpgradeable.__gap (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#77) shadows:
        - ContextUpgradeable.__gap (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#30)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variable-shadowing
INFO:Detectors:
BetweenNetwork.depositTokensAndPlatformFee(address,uint256) (BetweenNetwork.sol#260-269) performs a multiplication on the result of a division:
        -platformFee = _amount.mul(depositFee).div(MAX_PLATFORM_FEE).mul(multiplier) (BetweenNetwork.sol#261)
BetweenNetwork.withdrawTokensAndPlatformFee(address,uint256) (BetweenNetwork.sol#271-280) performs a multiplication on the result of a division:
        -platformFee = _amount.mul(withdrawFee).div(MAX_PLATFORM_FEE).mul(multiplier) (BetweenNetwork.sol#272)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#divide-before-multiply
INFO:Detectors:
BetweenNetwork.updateMultiplierReward(uint16) (BetweenNetwork.sol#426-429) should emit an event for:
        - multiplier = _newMultiplier (BetweenNetwork.sol#428)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-events-arithmetic
INFO:Detectors:
BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._governance (BetweenNetwork.sol#120) lacks a zero-check on :
                - governance = _governance (BetweenNetwork.sol#121)
BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._beneficiary (BetweenNetwork.sol#120) lacks a zero-check on :
                - beneficiary = _beneficiary (BetweenNetwork.sol#125)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Reentrancy in BetweenNetwork.tapIn(address,uint256) (BetweenNetwork.sol#222-235):
        External calls:
        - depositTokensAndPlatformFee(_homeERC20,_amount) (BetweenNetwork.sol#231)
                - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.
sol#92)
                - IERC20Upgradeable(_homeERC20).safeTransferFrom(msg.sender,beneficiary,platformFee) (BetweenNetwork.sol#263)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
                - IERC20Upgradeable(rewardToken).safeTransfer(msg.sender,platformFee) (BetweenNetwork.sol#266)
                - IERC20Upgradeable(_homeERC20).safeTransferFrom(msg.sender,address(this),_amount.sub(platformFee)) (BetweenNetwork.sol#268)
        External calls sending eth:
        - depositTokensAndPlatformFee(_homeERC20,_amount) (BetweenNetwork.sol#231)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
        State variables written after the call(s):
        - updateDeposit(lastDepositIndex,_amount,_homeERC20,trustedTokens[_homeERC20]._foreignERC20,block.number,block.timestamp) (BetweenNetwork.sol#233)
                - DepositWallets.push(msg.sender) (BetweenNetwork.sol#305)
        - lastDepositIndex = lastDepositIndex.add(1) (BetweenNetwork.sol#232)
Reentrancy in BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[]) (BetweenNetwork.sol#237-258):
        External calls:
        - withdrawTokensAndPlatformFee(_withdrawERC20,_amount) (BetweenNetwork.sol#254)
```

```
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
                - IERC20Upgradeable(rewardToken).safeTransfer(msg.sender,platformFee) (BetweenNetwork.sol#277)
                - IERC20Upgradeable(_withdrawERC20).safeTransfer(msg.sender,_amount.sub(platformFee)) (BetweenNetwork.sol#279)
        External calls sending eth:
        - withdrawTokensAndPlatformFee(_withdrawERC20,_amount) (BetweenNetwork.sol#254)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
        State variables written after the call(s):
        - updateWithdraw(_nonce,_amount,_withdrawERC20,_depositedERC20,_txHash) (BetweenNetwork.sol#256)
                - DepositeTxHashs.push(_txHash) (BetweenNetwork.sol#319)
        - updateWithdraw(_nonce,_amount,_withdrawERC20,_depositedERC20,_txHash) (BetweenNetwork.sol#256)
                - WithdrawalWallets.push(msg.sender) (BetweenNetwork.sol#318)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2
INFO:Detectors:
Reentrancy in BetweenNetwork.tapIn(address,uint256) (BetweenNetwork.sol#222-235):
        External calls:
        - depositTokensAndPlatformFee(_homeERC20,_amount) (BetweenNetwork.sol#231)
                - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.
sol#92)
                - IERC20Upgradeable(_homeERC20).safeTransferFrom(msg.sender,beneficiary,platformFee) (BetweenNetwork.sol#263)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
                - IERC20Upgradeable(rewardToken).safeTransfer(msg.sender,platformFee) (BetweenNetwork.sol#266)
                - IERC20Upgradeable(_homeERC20).safeTransferFrom(msg.sender,address(this),_amount.sub(platformFee)) (BetweenNetwork.sol#268)
        External calls sending eth:
        - depositTokensAndPlatformFee(_homeERC20,_amount) (BetweenNetwork.sol#231)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
        Event emitted after the call(s):
        - TapIn(msg.sender,_amount,_homeERC20,trustedTokens[_homeERC20]._foreignERC20,homeChainId,foreignChainId,block.number,block.timestamp,lastDepositIndex) (BetweenNetwo
rk.sol#234)
Reentrancy in BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[]) (BetweenNetwork.sol#237-258):
        External calls:
        - withdrawTokensAndPlatformFee(_withdrawERC20,_amount) (BetweenNetwork.sol#254)
                - returndata = address(token).functionCall(data,SafeERC20: low-level call failed) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.
sol#92)
                - IERC20Upgradeable(_withdrawERC20).safeTransfer(beneficiary,platformFee) (BetweenNetwork.sol#274)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
                - IERC20Upgradeable(rewardToken).safeTransfer(msg.sender,platformFee) (BetweenNetwork.sol#277)
                - IERC20Upgradeable(_withdrawERC20).safeTransfer(msg.sender,_amount.sub(platformFee)) (BetweenNetwork.sol#279)
        External calls sending eth:
        - withdrawTokensAndPlatformFee(_withdrawERC20,_amount) (BetweenNetwork.sol#254)
                - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
        Event emitted after the call(s):
        - TapOut(msg.sender,_amount,_withdrawERC20,_depositedERC20,_nonce,homeChainId,foreignChainId) (BetweenNetwork.sol#257)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3
INFO:Detectors:
AddressUpgradeable.isContract(address) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#26-36) uses assembly
```

```
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3
INFO:Detectors:
AddressUpgradeable.isContract(address) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#26-36) uses assembly
        - INLINE ASM (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#32-34)
AddressUpgradeable.verifyCallResult(bool,bytes,string) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#168-188) uses assembly
        - INLINE ASM (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#180-183)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
BetweenNetwork.createBridgePair(address,address) (BetweenNetwork.sol#207-220) compares to a boolean constant:
        -require(bool,string)(isBridgePaused == false,CreateBridgePair:: Bridge Paused, can not add new token) (BetweenNetwork.sol#208)
BetweenNetwork.tapIn(address,uint256) (BetweenNetwork.sol#222-235) compares to a boolean constant:
        -require(bool,string)(depositWallets[msg.sender]._isBanned == false,TapIn:: Your Wallet Banned) (BetweenNetwork.sol#225)
BetweenNetwork.tapIn(address,uint256) (BetweenNetwork.sol#222-235) compares to a boolean constant:
        -require(bool,string)(isTapInPaused == false,TapIn:: TapIn Paused, can not Deposit) (BetweenNetwork.sol#224)
BetweenNetwork.tapIn(address,uint256) (BetweenNetwork.sol#222-235) compares to a boolean constant:
        -require(bool,string)(isBridgePaused == false,TapIn:: Bridge Paused, can not Deposit) (BetweenNetwork.sol#223)
BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[]) (BetweenNetwork.sol#237-258) compares to a boolean constant:
        -require(bool,string)(withdrawalWallets[msg.sender]._isBanned == false,TapIn:: Your Wallet Banned) (BetweenNetwork.sol#240)
BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[]) (BetweenNetwork.sol#237-258) compares to a boolean constant:
        -require(bool,string)(isTapOutPaused == false,TapIn:: TapOut Paused, can not Withdrawn) (BetweenNetwork.sol#239)
BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[]) (BetweenNetwork.sol#237-258) compares to a boolean constant:
        -require(bool,string)(isBridgePaused == false,TapOut:: Bridge Paused, can not Withdrawn) (BetweenNetwork.sol#238)
BetweenNetwork.pauseTapIn() (BetweenNetwork.sol#355-359) compares to a boolean constant:
        -require(bool,string)(isTapInPaused == false,PauseTapIn:: TapIn already Paused) (BetweenNetwork.sol#356)
BetweenNetwork.unPauseTapIn() (BetweenNetwork.sol#361-365) compares to a boolean constant:
        -require(bool,string)(isTapInPaused == true,UnpauseTapIn:: TapIn already UnPaused) (BetweenNetwork.sol#362)
BetweenNetwork.pauseTapOut() (BetweenNetwork.sol#367-371) compares to a boolean constant:
        -require(bool,string)(isTapOutPaused == false,PauseTapOut:: TapOut already Paused) (BetweenNetwork.sol#368)
BetweenNetwork.unPauseTapOut() (BetweenNetwork.sol#373-377) compares to a boolean constant:
        -require(bool,string)(isTapOutPaused == true,UnpauseTapOut:: TapOut already UnPaused) (BetweenNetwork.sol#374)
BetweenNetwork.pauseBridge() (BetweenNetwork.sol#408-412) compares to a boolean constant:
        -require(bool,string)(isBridgePaused == false,PauseBridge:: Bridge is already Paused) (BetweenNetwork.sol#409)
BetweenNetwork.unPauseBridge() (BetweenNetwork.sol#414-418) compares to a boolean constant:
        -require(bool,string)(isBridgePaused == true,UnPauseBridge:: Bridge is already Unpaused) (BetweenNetwork.sol#415)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#boolean-equality
INFO:Detectors:
Different versions of Solidity is used:
        - Version used: ['0.8.4', '^0.8.0']
        - 0.8.4 (BetweenNetwork.sol#2)
        - v2 (BetweenNetwork.sol#3)
        - 0.8.4 (Libraries/AddressArrayUtils.sol#2)
        - 0.8.4 (Libraries/StringLibrary.sol#2)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#3)
```

```
        -require(bool,string)(isBridgePaused == true,UnPauseBridge:: Bridge is already Unpaused) (BetweenNetwork.sol#415)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#boolean-equality
INFO:Detectors:
Different versions of Solidity is used:
        - Version used: ['0.8.4', '^0.8.0']
        - 0.8.4 (BetweenNetwork.sol#2)
        - v2 (BetweenNetwork.sol#3)
        - 0.8.4 (Libraries/AddressArrayUtils.sol#2)
        - 0.8.4 (Libraries/StringLibrary.sol#2)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/token/ERC20/IERC20Upgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/utils/StringsUpgradeable.sol#3)
        - ^0.8.0 (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#3)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:
AddressUpgradeable.functionCall(address,bytes) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#79-81) is never used and should be removed
AddressUpgradeable.functionCallWithValue(address,bytes,uint256) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#108-114) is never used and should be removed
AddressUpgradeable.functionStaticCall(address,bytes) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#141-143) is never used and should be removed
AddressUpgradeable.functionStaticCall(address,bytes,string) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#151-160) is never used and should be removed
AddressUpgradeable.sendValue(address,uint256) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#54-59) is never used and should be removed
ContextUpgradeable.__Context_init() (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#17-19) is never used and should be removed
ContextUpgradeable.__Context_init_unchained() (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#21-22) is never used and should be removed
ContextUpgradeable._msgData() (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#27-29) is never used and should be removed
OwnableUpgradeable.__Ownable_init() (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#28-31) is never used and should be removed
OwnableUpgradeable.__Ownable_init_unchained() (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#33-35) is never used and should be removed
ReentrancyGuardUpgradeable.__ReentrancyGuard_init() (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#39-41) is never used and should be removed
ReentrancyGuardUpgradeable.__ReentrancyGuard_init_unchained() (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#43-45) is never used and should be
removed
SafeERC20Upgradeable.safeApprove(IERC20Upgradeable,address,uint256) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol#44-57) is never used and s
hould be removed
SafeERC20Upgradeable.safeDecreaseAllowance(IERC20Upgradeable,address,uint256) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol#68-79) is never
used and should be removed
SafeERC20Upgradeable.safeIncreaseAllowance(IERC20Upgradeable,address,uint256) (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol#59-66) is never
used and should be removed
SafeMathUpgradeable.div(uint256,uint256,string) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#190-199) is never used and should be removed
SafeMathUpgradeable.mod(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#150-152) is never used and should be removed
SafeMathUpgradeable.mod(uint256,uint256,string) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#216-225) is never used and should be removed
SafeMathUpgradeable.sub(uint256,uint256,string) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#167-176) is never used and should be removed
SafeMathUpgradeable.tryAdd(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#21-27) is never used and should be removed
SafeMathUpgradeable.tryDiv(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#63-68) is never used and should be removed
```

```
SafeMathUpgradeable.tryAdd(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#21-27) is never used and should be removed
SafeMathUpgradeable.tryDiv(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#63-68) is never used and should be removed
SafeMathUpgradeable.tryMod(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#75-80) is never used and should be removed
SafeMathUpgradeable.tryMul(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#46-56) is never used and should be removed
SafeMathUpgradeable.trySub(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#34-39) is never used and should be removed
StringLibrary.append(string,string) (Libraries/StringLibrary.sol#8-16) is never used and should be removed
StringLibrary.append(string,string,string) (Libraries/StringLibrary.sol#18-28) is never used and should be removed
StringLibrary.recover(string,uint8,bytes32,bytes32) (Libraries/StringLibrary.sol#30-39) is never used and should be removed
StringsUpgradeable.toHexString(uint256) (openzeppelin/contracts-upgradeable/utils/StringsUpgradeable.sol#39-50) is never used and should be removed
StringsUpgradeable.toHexString(uint256,uint256) (openzeppelin/contracts-upgradeable/utils/StringsUpgradeable.sol#55-65) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code
INFO:Detectors:
Pragma version0.8.4 (BetweenNetwork.sol#2) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
Pragma version0.8.4 (Libraries/AddressArrayUtils.sol#2) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
Pragma version0.8.4 (Libraries/StringLibrary.sol#2) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7
.6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7
.6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with
0.6.12/0.7.6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/token/ERC20/IERC20Upgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12
/0.7.6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying wi
th 0.6.12/0.7.6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.
6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.
6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/utils/StringsUpgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.
6
Pragma version^0.8.0 (openzeppelin/contracts-upgradeable/utils/math/SafeMathUpgradeable.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.1
2/0.7.6
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Low level call in AddressUpgradeable.sendValue(address,uint256) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#54-59):
        - (success) = recipient.call{value: amount}() (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#57)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#122-133):
        - (success,returndata) = target.call{value: value}(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#131)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#151-160):
        - (success,returndata) = target.staticcall(data) (openzeppelin/contracts-upgradeable/utils/AddressUpgradeable.sol#158)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
```

```
INFO:Detectors:
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._governance (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._validators (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._required (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._beneficiary (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._depositFee (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._withdrawFee (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._listingFee (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._foreignChainId (BetweenNetwork.sol#120) is not in mixedCase
Parameter BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)._brdigeName (BetweenNetwork.sol#120) is not in mixedCase
Function BetweenNetwork.MultiSigWallet(address[],uint256) (BetweenNetwork.sol#173-180) is not in mixedCase
Parameter BetweenNetwork.MultiSigWallet(address[],uint256)._validator (BetweenNetwork.sol#173) is not in mixedCase
Parameter BetweenNetwork.MultiSigWallet(address[],uint256)._required (BetweenNetwork.sol#173) is not in mixedCase
Parameter BetweenNetwork.changeRequirement(uint256)._required (BetweenNetwork.sol#201) is not in mixedCase
Parameter BetweenNetwork.createBridgePair(address,address)._homeERC20 (BetweenNetwork.sol#207) is not in mixedCase
Parameter BetweenNetwork.createBridgePair(address,address)._foreignERC20 (BetweenNetwork.sol#207) is not in mixedCase
Parameter BetweenNetwork.tapIn(address,uint256)._homeERC20 (BetweenNetwork.sol#222) is not in mixedCase
Parameter BetweenNetwork.tapIn(address,uint256)._amount (BetweenNetwork.sol#222) is not in mixedCase
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._amount (BetweenNetwork.sol#237) is not in mixedCase
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._withdrawERC20 (BetweenNetwork.sol#237) is not in mixedCas
e
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._depositedERC20 (BetweenNetwork.sol#237) is not in mixedCa
se
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._chainID (BetweenNetwork.sol#237) is not in mixedCase
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._nonce (BetweenNetwork.sol#237) is not in mixedCase
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._txHash (BetweenNetwork.sol#237) is not in mixedCase
Parameter BetweenNetwork.tapOut(uint256,address,address,uint256[2],uint256,bytes32,BetweenNetwork.Sig[],address[])._validators (BetweenNetwork.sol#237) is not in mixedCase
Parameter BetweenNetwork.depositTokensAndPlatformFee(address,uint256)._homeERC20 (BetweenNetwork.sol#260) is not in mixedCase
Parameter BetweenNetwork.depositTokensAndPlatformFee(address,uint256)._amount (BetweenNetwork.sol#260) is not in mixedCase
Parameter BetweenNetwork.withdrawTokensAndPlatformFee(address,uint256)._withdrawERC20 (BetweenNetwork.sol#271) is not in mixedCase
Parameter BetweenNetwork.withdrawTokensAndPlatformFee(address,uint256)._amount (BetweenNetwork.sol#271) is not in mixedCase
Parameter BetweenNetwork.updatePair(uint256,address,address)._index (BetweenNetwork.sol#282) is not in mixedCase
Parameter BetweenNetwork.updatePair(uint256,address,address)._homeERC20 (BetweenNetwork.sol#282) is not in mixedCase
Parameter BetweenNetwork.updatePair(uint256,address,address)._foreignERC20 (BetweenNetwork.sol#282) is not in mixedCase
Parameter BetweenNetwork.updateDeposit(uint256,uint256,address,address,uint256,uint256)._index (BetweenNetwork.sol#295) is not in mixedCase
Parameter BetweenNetwork.updateDeposit(uint256,uint256,address,address,uint256,uint256)._amount (BetweenNetwork.sol#295) is not in mixedCase
Parameter BetweenNetwork.updateDeposit(uint256,uint256,address,address,uint256,uint256)._homeERC20 (BetweenNetwork.sol#295) is not in mixedCase
Parameter BetweenNetwork.updateDeposit(uint256,uint256,address,address,uint256,uint256)._foreignERC20 (BetweenNetwork.sol#295) is not in mixedCase
Parameter BetweenNetwork.updateDeposit(uint256,uint256,address,address,uint256,uint256)._depositedBlock (BetweenNetwork.sol#295) is not in mixedCase
Parameter BetweenNetwork.updateDeposit(uint256,uint256,address,address,uint256,uint256)._depositedTimeStamp (BetweenNetwork.sol#295) is not in mixedCase
Parameter BetweenNetwork.updateWithdraw(uint256,uint256,address,address,bytes32)._nonce (BetweenNetwork.sol#308) is not in mixedCase
Parameter BetweenNetwork.updateWithdraw(uint256,uint256,address,address,bytes32)._amount (BetweenNetwork.sol#308) is not in mixedCase
Parameter BetweenNetwork.updateWithdraw(uint256,uint256,address,address,bytes32)._withdrawERC20 (BetweenNetwork.sol#308) is not in mixedCase
Parameter BetweenNetwork.updateWithdraw(uint256,uint256,address,address,bytes32)._depositedERC20 (BetweenNetwork.sol#308) is not in mixedCase
Parameter BetweenNetwork.updateWithdraw(uint256,uint256,address,address,bytes32)._txHash (BetweenNetwork.sol#308) is not in mixedCase
```

```
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._amount (BetweenNetwork.sol#322) is not in mixed
Case
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._withdraERC20 (BetweenNetwork.sol#322) is not in
 mixedCase
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._depositedERC20 (BetweenNetwork.sol#322) is not
in mixedCase
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._homeChainId (BetweenNetwork.sol#322) is not in
mixedCase
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._foreignChainId (BetweenNetwork.sol#322) is not
in mixedCase
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._nonce (BetweenNetwork.sol#322) is not in mixedC
ase
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._txHash (BetweenNetwork.sol#322) is not in mixed
Case
Parameter BetweenNetwork.verifySignature(uint256,address,address,uint256,uint256,uint256,bytes32,BetweenNetwork.Sig,address)._validator (BetweenNetwork.sol#322) is not in mi
xedCase
Parameter BetweenNetwork.suspendToken(address)._erc20Token (BetweenNetwork.sol#327) is not in mixedCase
Parameter BetweenNetwork.activeSuspendToken(address)._erc20Token (BetweenNetwork.sol#334) is not in mixedCase
Parameter BetweenNetwork.banUser(address)._wallet (BetweenNetwork.sol#341) is not in mixedCase
Parameter BetweenNetwork.unBanUser(address)._wallet (BetweenNetwork.sol#348) is not in mixedCase
Parameter BetweenNetwork.updateListingFee(uint256)._newListingFee (BetweenNetwork.sol#379) is not in mixedCase
Parameter BetweenNetwork.updateDepositFee(uint256)._newDepositFee (BetweenNetwork.sol#386) is not in mixedCase
Parameter BetweenNetwork.updateWithdrawFee(uint256)._newWithdrawFee (BetweenNetwork.sol#394) is not in mixedCase
Parameter BetweenNetwork.updateBeneficiary(address)._newBeneficiary (BetweenNetwork.sol#402) is not in mixedCase
Parameter BetweenNetwork.updateRewardToken(address)._newRewardToken (BetweenNetwork.sol#420) is not in mixedCase
Parameter BetweenNetwork.updateMultiplierReward(uint16)._newMultiplier (BetweenNetwork.sol#426) is not in mixedCase
Parameter BetweenNetwork.validatorSignMessage(address,uint256,address,address,uint256,bytes32)._userWallet (BetweenNetwork.sol#436) is not in mixedCase
Parameter BetweenNetwork.validatorSignMessage(address,uint256,address,address,uint256,bytes32)._amount (BetweenNetwork.sol#436) is not in mixedCase
Parameter BetweenNetwork.validatorSignMessage(address,uint256,address,address,uint256,bytes32)._withdraERC20 (BetweenNetwork.sol#436) is not in mixedCase
Parameter BetweenNetwork.validatorSignMessage(address,uint256,address,address,uint256,bytes32)._depositedERC20 (BetweenNetwork.sol#436) is not in mixedCase
Parameter BetweenNetwork.validatorSignMessage(address,uint256,address,address,uint256,bytes32)._nonce (BetweenNetwork.sol#436) is not in mixedCase
Parameter BetweenNetwork.validatorSignMessage(address,uint256,address,address,uint256,bytes32)._txHash (BetweenNetwork.sol#436) is not in mixedCase
Variable BetweenNetwork.MAX_PLATFORM_FEE (BetweenNetwork.sol#30) is not in mixedCase
Variable BetweenNetwork.MAX_VALIDATOR_COUNT (BetweenNetwork.sol#31) is not in mixedCase
Variable BetweenNetwork.BRIDGENAME (BetweenNetwork.sol#36) is not in mixedCase
Variable BetweenNetwork.TrustedTokens (BetweenNetwork.sol#61) is not in mixedCase
Variable BetweenNetwork.DepositWallets (BetweenNetwork.sol#76) is not in mixedCase
Variable BetweenNetwork.WithdrawalWallets (BetweenNetwork.sol#92) is not in mixedCase
Variable BetweenNetwork.DeposiTeTxHashs (BetweenNetwork.sol#93) is not in mixedCase
Parameter AddressArrayUtils.hasDuplicate(address[]).A (Libraries/AddressArrayUtils.sol#11) is not in mixedCase
Parameter StringLibrary.append(string,string)._a (Libraries/StringLibrary.sol#8) is not in mixedCase
Parameter StringLibrary.append(string,string)._b (Libraries/StringLibrary.sol#8) is not in mixedCase
Parameter StringLibrary.append(string,string,string)._a (Libraries/StringLibrary.sol#18) is not in mixedCase
```

```
Parameter StringLibrary.append(string,string,string)._c (Libraries/StringLibrary.sol#18) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._ba (Libraries/StringLibrary.sol#41) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._bb (Libraries/StringLibrary.sol#41) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._bc (Libraries/StringLibrary.sol#41) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._bd (Libraries/StringLibrary.sol#41) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._be (Libraries/StringLibrary.sol#41) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._bf (Libraries/StringLibrary.sol#41) is not in mixedCase
Parameter StringLibrary.concat(bytes,bytes,bytes,bytes,bytes,bytes,bytes)._bg (Libraries/StringLibrary.sol#41) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#28-31) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#33-35) is not in mixedCase
Variable OwnableUpgradeable.__gap (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#77) is not in mixedCase
Function ReentrancyGuardUpgradeable.__ReentrancyGuard_init() (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#39-41) is not in mixedCase
Function ReentrancyGuardUpgradeable.__ReentrancyGuard_init_unchained() (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#43-45) is not in mixedCase
Variable ReentrancyGuardUpgradeable.__gap (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#67) is not in mixedCase
Function ContextUpgradeable.__Context_init() (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#17-19) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#21-22) is not in mixedCase
Variable ContextUpgradeable.__gap (openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#30) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Reentrancy in BetweenNetwork.createBridgePair(address,address) (BetweenNetwork.sol#207-220):
        External calls:
        - address(beneficiary).transfer(listingFee) (BetweenNetwork.sol#215)
        State variables written after the call(s):
        - updatePair(lastPairIndex,_homeERC20,_foreignERC20) (BetweenNetwork.sol#218)
                - TrustedTokens.push(_homeERC20) (BetweenNetwork.sol#292)
        - lastPairIndex = lastPairIndex.add(1) (BetweenNetwork.sol#217)
        - updatePair(lastPairIndex,_homeERC20,_foreignERC20) (BetweenNetwork.sol#218)
                - trustedTokens[_homeERC20]._index = _index (BetweenNetwork.sol#283)
                - trustedTokens[_homeERC20]._homeERC20 = _homeERC20 (BetweenNetwork.sol#284)
                - trustedTokens[_homeERC20]._foreignERC20 = _foreignERC20 (BetweenNetwork.sol#285)
                - trustedTokens[_homeERC20]._homeChainId = homeChainId (BetweenNetwork.sol#286)
                - trustedTokens[_homeERC20]._foreignChainId = foreignChainId (BetweenNetwork.sol#287)
                - trustedTokens[_homeERC20]._listedBy = msg.sender (BetweenNetwork.sol#288)
                - trustedTokens[_homeERC20]._listedBlock = block.number (BetweenNetwork.sol#289)
                - trustedTokens[_homeERC20]._suspendedBlock = 0 (BetweenNetwork.sol#290)
                - trustedTokens[_homeERC20]._isActive = 1 (BetweenNetwork.sol#291)
        Event emitted after the call(s):
        - BridgePairCreated(msg.sender,_homeERC20,homeChainId,_foreignERC20,foreignChainId,msg.sender,block.number,block.timestamp,lastPairIndex) (BetweenNetwork.sol#219)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-4
INFO:Detectors:
ReentrancyGuardUpgradeable.__gap (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#67) is never used in BetweenNetwork (BetweenNetwork.sol#13-440)
BetweenNetwork.withdrawalWallet (BetweenNetwork.sol#90) is never used in BetweenNetwork (BetweenNetwork.sol#13-440)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-state-variables
INFO:Detectors:
```

```
INFO:Detectors:
ReentrancyGuardUpgradeable.__gap (openzeppelin/contracts-upgradeable/security/ReentrancyGuardUpgradeable.sol#67) is never used in BetweenNetwork (BetweenNetwork.sol#13-440)
BetweenNetwork.withdrawalWallet (BetweenNetwork.sol#90) is never used in BetweenNetwork (BetweenNetwork.sol#13-440)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-state-variables
INFO:Detectors:
initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string) should be declared external:
        - BetweenNetwork.initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string) (BetweenNetwork.sol#120-138)
renounceOwnership() should be declared external:
        - OwnableUpgradeable.renounceOwnership() (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#59-61)
transferOwnership(address) should be declared external:
        - OwnableUpgradeable.transferOwnership(address) (openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#67-70)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#public-function-that-could-be-declared-external
```

Mythril

```
==== Use of tx.origin ====
SWC ID: 111
Severity: Medium
Contract: 0x345424Fe54FF70b9543D6f208a56EeB4dA4941eE
Function name: tapIn(address,uint256)
PC address: 4000
Estimated Gas Usage: 1153 - 1248
Use of tx.origin is deprecated.
The smart contract retrieves the transaction origin (tx.origin) using msg.origin. Use of msg.origin is deprecated and the instruction may be removed in the  future. Use msg.
sender instead.
See also: https://solidity.readthedocs.io/en/develop/security-considerations.html#tx-origin
--------------------
Initial State:

Account: [ATTACKER], balance: 0x0, nonce:0, storage:{}
Account: [SOMEGUY], balance: 0x0, nonce:0, storage:{}

Transaction Sequence:

Caller: [SOMEGUY], function: tapIn(address,uint256), txdata: 0x0d599186000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000, value: 0x0

==== Use of tx.origin ====
SWC ID: 111
Severity: Medium
Contract: 0x345424Fe54FF70b9543D6f208a56EeB4dA4941eE
Function name: createBridgePair(address,address)
PC address: 9269
Estimated Gas Usage: 1165 - 1260
Use of tx.origin is deprecated.
The smart contract retrieves the transaction origin (tx.origin) using msg.origin. Use of msg.origin is deprecated and the instruction may be removed in the  future. Use msg.
sender instead.
See also: https://solidity.readthedocs.io/en/develop/security-considerations.html#tx-origin
--------------------
Initial State:

Account: [ATTACKER], balance: 0x1, nonce:0, storage:{}
Account: [SOMEGUY], balance: 0x0, nonce:0, storage:{}

Transaction Sequence:

Caller: [ATTACKER], function: createBridgePair(address,address), txdata: 0x8a66ac890000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000, value: 0x0
```

```
==== Potential denial-of-service if block gas limit is reached ====
SWC ID: 128
Severity: Low
Contract: 0x345424Fe54FF70b9543D6f208a56EeB4dA4941eE
Function name: initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string)
PC address: 18338
Estimated Gas Usage: 18749 - 81904
Potential denial-of-service if block gas limit is reached.
A storage modification is executed in a loop. Be aware that the transaction may fail to execute if the loop is unbounded and the necessary gas exceeds the block gas limit.
--------------------
Initial State:

Account: [ATTACKER], balance: 0x1, nonce:0, storage:{}
Account: [SOMEGUY], balance: 0x0, nonce:0, storage:{}

Transaction Sequence:

Caller: [ATTACKER], function: initialize(address,address[],uint256,address,uint256,uint256,uint256,uint256,string), txdata: 0x0388ba270000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000600000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000010000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000e, value: 0x0
```

SOLHINT LINTER

```
BetweenNetwork.sol
    2:1     error    Compiler version 0.8.4 does not satisfy the ^0.5.8 semver requirement    compiler-version
   13:1     warning  Contract has 30 states declarations but allowed no more than 15          max-states-count
   30:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
   31:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
   36:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
   61:5     warning  Explicitly mark visibility of state                                      state-visibility
   61:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
   76:5     warning  Explicitly mark visibility of state                                      state-visibility
   76:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
   90:5     warning  Explicitly mark visibility of state                                      state-visibility
   92:5     warning  Explicitly mark visibility of state                                      state-visibility
   92:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
   93:5     warning  Variable name must be in mixedCase                                       var-name-mixedcase
  141:9     warning  Error message for require is too long                                    reason-string
  141:56    warning  Avoid to use tx.origin                                                   avoid-tx-origin
  151:9     warning  Provide an error message for require                                     reason-string
  156:9     warning  Provide an error message for require                                     reason-string
  161:9     warning  Provide an error message for require                                     reason-string
  166:9     warning  Provide an error message for require                                     reason-string
  173:5     warning  Function name must be in mixedCase                                       func-name-mixedcase
  175:13    warning  Provide an error message for require                                     reason-string
  202:9     warning  Error message for require is too long                                    reason-string
  208:9     warning  Error message for require is too long                                    reason-string
  209:9     warning  Error message for require is too long                                    reason-string
  210:9     warning  Error message for require is too long                                    reason-string
  211:9     warning  Error message for require is too long                                    reason-string
  212:9     warning  Error message for require is too long                                    reason-string
  214:13    warning  Error message for require is too long                                    reason-string
  217:9     warning  Possible reentrancy vulnerabilities. Avoid state changes after transfer  reentrancy
  219:126   warning  Avoid to make time-based decisions in your business logic                 not-rely-on-time
  223:9     warning  Error message for require is too long                                    reason-string
  224:9     warning  Error message for require is too long                                    reason-string
  226:9     warning  Error message for require is too long                                    reason-string
  227:9     warning  Error message for require is too long                                    reason-string
  233:117   warning  Avoid to make time-based decisions in your business logic                 not-rely-on-time
  234:137   warning  Avoid to make time-based decisions in your business logic                 not-rely-on-time
  238:9     warning  Error message for require is too long                                    reason-string
  239:9     warning  Error message for require is too long                                    reason-string
  241:9     warning  Error message for require is too long                                    reason-string
```

```
 223:9    warning  Error message for require is too long                                reason-string
 224:9    warning  Error message for require is too long                                reason-string
 226:9    warning  Error message for require is too long                                reason-string
 227:9    warning  Error message for require is too long                                reason-string
 233:117  warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 234:137  warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 238:9    warning  Error message for require is too long                                reason-string
 239:9    warning  Error message for require is too long                                reason-string
 241:9    warning  Error message for require is too long                                reason-string
 242:9    warning  Error message for require is too long                                reason-string
 243:9    warning  Error message for require is too long                                reason-string
 244:9    warning  Error message for require is too long                                reason-string
 248:9    warning  Error message for require is too long                                reason-string
 250:9    warning  Error message for require is too long                                reason-string
 252:13   warning  Error message for require is too long                                reason-string
 328:9    warning  Error message for require is too long                                reason-string
 331:40   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 335:9    warning  Error message for require is too long                                reason-string
 338:42   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 342:9    warning  Error message for require is too long                                reason-string
 345:31   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 349:9    warning  Error message for require is too long                                reason-string
 352:33   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 356:9    warning  Error message for require is too long                                reason-string
 358:25   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 362:9    warning  Error message for require is too long                                reason-string
 364:27   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 368:9    warning  Error message for require is too long                                reason-string
 370:26   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 374:9    warning  Error message for require is too long                                reason-string
 376:28   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 380:9    warning  Error message for require is too long                                reason-string
 381:9    warning  Error message for require is too long                                reason-string
 387:9    warning  Error message for require is too long                                reason-string
 388:9    warning  Error message for require is too long                                reason-string
 389:9    warning  Error message for require is too long                                reason-string
 395:9    warning  Error message for require is too long                                reason-string
 396:9    warning  Error message for require is too long                                reason-string
 397:9    warning  Error message for require is too long                                reason-string
 403:9    warning  Error message for require is too long                                reason-string
 409:9    warning  Error message for require is too long                                reason-string
 411:26   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 415:9    warning  Error message for require is too long                                reason-string
 417:28   warning  Avoid to make time-based decisions in your business logic            not-rely-on-time
 421:9    warning  Error message for require is too long                                reason-string
```

Disclaimer

Quillhash audit is not a security warranty, investment advice, or an endorsement of the BetweenNetwork platform. This audit does not provide a security or correctness guarantee of the audited smart contracts. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them. Securing smart contracts is a multistep process. One audit cannot be considered enough. We recommend that the BetweenNetwork Team put in place a bug bounty program to encourage further analysis of the smart contract by other third parties.

Closing Summary

In this report, we have considered the security of the BetweenNetwork platform. We performed our audit according to the procedure described above.

The audit showed several high, medium, low, and informational severity issues. In the end, the majority of the issues were fixed or acknowledged by the Auditee.

between.network

QuillAudits

Canada, India, Singapore and United Kingdom

audits.quillhash.com

audits@quillhash.com